

PROCESRAPPORT

IoT og Embedded case - Robotplæneklipper

Gruppe: Adem Akat og Mark

Uddannelse: AarhusTech

Projekt: IoT / Embedded prototype

Aflevering: September 2026

Rapporten beskriver vores arbejdsproces, tekniske valg, datastrøm, test, fejlfinding og de erfaringer vi har fået under udviklingen af prototypen.

1. Problemformulering

I dette projekt har vi arbejdet med at udvikle en fungerende prototype af en autonom robotplæneklipper som en IoT- og embedded-løsning. Vi havde ikke mulighed for at bygge en rigtig robotplæneklipper, og derfor brugte vi skolens 2WD-robotplatform med to DC-motorer og en Raspberry Pi som hovedenhed.

Formålet var at få robotten til at kunne bevæge sig selvstændigt i et defineret testområde. Robotten skulle kunne køre frem og tilbage, dreje og stoppe, registrere sin bevægelse og reagere på forhindringer. Til dette brugte vi hjulencodere og en LD06 LiDAR.

En vigtig del af opgaven var også IoT-kommunikation. Robotten skulle kunne sende relevante målinger og status trådløst til en central løsning og kunne modtage kommandoer den anden vej. Vi valgte Wi-Fi og MQTT til kommunikationen. Vi arbejdede blandt andet med status, position, forhindringer, kommandoer og batteriniveau.

Vi tilføjede også en Arduino Uno R4 WiFi til batteriovervågning. Arduinoen måler spændingen på motorbatteriet gennem en spændingsdeler, omregner målingen til en forståelig batteriprocent og sender procenten via Wi-Fi og MQTT til resten af systemet.

Problemformulering: Hvordan kan vi udvikle en prototype af en autonom robotplæneklipper, som kan navigere i et afgrænset område, registrere og reagere på forhindringer samt sende og modtage relevante data og kommandoer gennem en trådløs IoT-løsning?

2. Planlægning

Arbejdsfordeling og tidsplan

Projektperioden var cirka to uger. Før implementeringen lavede vi en kravspecifikation og en MoSCoW-prioritering. Kravspecifikationen blev godkendt af underviseren, og vi prioriterede Must-have funktionerne først, fordi opgaven handler om en fungerende prototype og ikke et færdigt kommercielt produkt.

Den oprindelige plan var, at vi skulle arbejde parallelt med forskellige dele og løbende integrere dem. Mark var syg i den første uge. Adem arbejdede derfor videre med blandt andet Raspberry Pi-opsætning, motorstyring, encodere og de første integrationstests. Da Mark kom tilbage i anden uge, fordelte vi arbejdet igen, så flere dele kunne udvikles parallelt. Mark arbejdede især videre med LiDAR, navigation og netværkskommunikation, mens Adem blandt andet arbejdede med integration, test og batteriovervågning med Arduino Uno R4 WiFi.

Sygdommen betød, at den oprindelige tidsplan måtte justeres. Vi valgte derfor at holde fokus på Must-have krav og undlod at prioritere større ekstra funktioner som GPS og automatisk ladestation, før den grundlæggende kæde fungerede.

Valg af hardware

Prototypen består af en 2WD-platform med to DC-gearmotorer. Motorerne styres gennem en L9110S H-bridge, så Raspberry Pi'en kan styre frem, tilbage, venstre, højre og stop. Raspberry Pi 3 med Raspberry Pi OS Lite fungerer som robotens hovedcomputer.

To optiske hjulencodere bruges til at registrere bevægelse. Ved at kombinere antal pulser med hjuldiameteren kan vi beregne en tilnærmet kørt afstand og bruge den til relativ position. En LD06 LiDAR bruges til afstandsmåling og registrering af forhindringer foran robotten.

Til batteriovervågning bruger vi en Arduino Uno R4 WiFi. Motorforsyningen består af fire AA-batterier. To 10 kΩ-modstande danner en spændingsdeler, så Arduinoens A0-indgang kan måle batterispændingen. Arduinoen får under udvikling strøm via USB fra en laptop.

Valg af software og kommunikation

På Raspberry Pi'en har vi arbejdet med C++ til den samlede robotløsning og mindre Python- og shell-scripts til isolerede hardwaretests. Projektet er opdelt i moduler til blandt andet motor, encoder, LiDAR,

navigation, konfiguration og netværk. Arduino Uno R4 er programmeret i C++ gennem Visual Studio Code og PlatformIO.

Vi valgte Wi-Fi og MQTT til den trådløse kommunikation. MQTT passer til IoT, fordi enheder kan publicere og abonnere på bestemte topics. I projektet bruges blandt andet robot/commands, robot/telemetry og robot/battery. Git og GitHub bruges til versionsstyring, kodebase og løbende dokumentation.

Grov arbejdsrækkefølge

Trin	Fokus
1	Motorstyring
2	Encodere og afstand
3	LiDAR og forhindringer
4	Autonom reaktion og navigation
5	Wi-Fi/MQTT
6	Batteriovervågning
7	Server/database og samlet integration
8	Test, fejlfinding og dokumentation

3. Udførelse

Raspberry Pi og Git

Vi startede med at klargøre Raspberry Pi'en med Raspberry Pi OS Lite 64-bit, Wi-Fi og SSH. Vi oplevede tidligt en mindre fejl, fordi vi først forsøgte at forbinde til raspberrypi.local, men vores valgte hostname var robotpi. Da vi brugte robotpi.local, virkede forbindelsen.

Git var ikke installeret fra starten og blev derfor installeret. Den første lokale projektmappe var heller ikke et Git-repository. Vi bevarede den som backup og klonede derefter GitHub-repoet korrekt. Undervejs havde vi også en GitHub-token, der ikke havde rettigheder til det nye repo. Det blev løst ved at oprette en token med de rigtige repository-rettigheder og konfigurere Git-identitet på Raspberry Pi'en.

Motorstyring

Motorerne blev tilsluttet L9110S-driveren. Vi testede GPIO-kombinationerne manuelt og fandt de korrekte retninger for frem, tilbage og drejning. Under testen opdagede vi, at en GPIO forbliver HIGH, indtil den ændres igen. En motor kunne derfor fortsætte med at køre, hvis vi ikke aktivt satte udgangen LOW. Det gjorde stopfunktionen vigtig som sikkerhedsdel.

Vi havde også forvirring omkring retningen på den højre motor. Vi testede motorerne længere ad gangen og definerede en fast 'fremad'-retning set ovenfra. Efter justering fungerede frem, tilbage, venstre, højre og stop.

Encodere og afstand

Venstre encoder blev koblet til GPIO24 og højre encoder til GPIO25. Den første sensor på GPIO24 gav konstant LOW, selv når encoderhjulet bevægede sig. Efter systematisk test og udskiftning viste det sig, at sensoren var defekt. Den nye sensor skiftede korrekt mellem HIGH og LOW.

I Python fik vi også fejlen PinInvalidState, fordi et floating input ikke havde en defineret active_state. Det blev løst ved at angive active_state=True. Sensorerne blev derefter kalibreret. Den endelige kalibrering var cirka 22 pulser pr. omdrejning på begge hjul, og hjuldiameteren var cirka 6,5 cm.

I den første afstandstest ventede programmet på, at begge hjul nåede målet, før motorerne stoppede. Det hurtigste hjul kunne derfor overskyde. Vi ændrede logikken, så hver motor stoppede individuelt, når dens

egen encoder nåede målet. Ved en test med et mål på 50 cm registrerede begge sider omkring 54 pulser, svarende til cirka 50,1 cm beregnet afstand.

LiDAR og autonom integration

LD06 LiDAR blev brugt til at registrere forhindringer foran robotten. Under den afsluttende test fandt vi en fejl i vores første fortolkning af LD06-pakkerne: startvinklen blev læst fra de forkerte bytes, og slutvinklen blev også læst forkert. Efter rettelsen brugte parseren byte 4-5 til startvinkel, byte 42-43 til slutvinkel og byte 6-41 til de 12 målepunkter.

Efter parserrettelsen kalibrerede vi LiDAR'en på den fysiske prototype. Frontområdet blev sat til cirka 130-175 grader. Meget korte ekkoer fra robotten selv blev filtreret væk, og der kræves mindst to gyldige målepunkter, før en forhindring accepteres. I den afsluttende test blev forhindringer omkring 17-20 cm registreret med 5-7 hits. Robotten stoppede, bakkede, drejede og fortsatte i en ny retning. Dermed blev FK4 demonstreret i den samlede C++-løsning.

MQTT og netværk

Til den trådløse del blev MQTT valgt. Robotkoden arbejder med en MQTT-broker på port 1883 og topics til blandt andet kommandoer, telemetry og batteri. Robotten kan abonnere på robot/commands, mens status og målinger kan publiceres som telemetry.

Under test af batteritopic på Raspberry Pi'en fandtes kommandoen `mosquitto_sub` ikke, selv om MQTT-biblioteket blev brugt af C++-projektet. Vi installerede derfor pakken `mosquitto-clients`. Derefter kunne Raspberry Pi'en abonnere på robot/battery og modtage batteriprocenter fra Arduinoen.

Batteriovervågning med Uno R4 WiFi

Batteriovervågningen blev lavet som en separat del med Arduino Uno R4 WiFi. Først målte vi kun råværdien fra A0. Den første test gav næsten ingen ændring, selv om motorbatteriet var tændt. Fejlsøgningen viste, at vi ved en fejl havde taget målingen fra motor A/B-udgangen på driveren i stedet for direkte fra batteriforsyningen.

Da målepunktet blev flyttet til batteriets forsyning, lå A0 stabilt omkring 616-618. Med spændingsdeleren blev dette omregnet til cirka 6,0 V. Vi lavede derefter en simpel prototypeberegning fra spænding til 0-100 % batteriniveau. Beregningen er en tilnærmelse og ikke en præcis batterimodel.

Vi testede low-battery logikken ved midlertidigt at simulere 15 %. Arduinoen viste korrekt WARNING: LOW BATTERY. Testlinjen blev derefter fjernet igen.

Næste trin var at koble batteridelen til IoT-systemet. Uno R4 blev forbundet til skolens Wi-Fi og MQTT-broderen og publicerede batteriprocenten på robot/battery. Raspberry Pi'en modtog efterfølgende værdier omkring 78-84 %. Dermed blev det bevist, at kæden fra fysisk batterimåling til trådløs MQTT-data fungerede.

Ved den første MQTT-opstart fra Arduinoen kom der kortvarigt `state=-2`, men næste forbindelsesforsøg lykkedes automatisk. Dette viste også værdien af retry-logik ved netværksopstart.

Samlet integration og afsluttende test

Den tidligere buildfejl omkring `send_telemetry` og `battery_pct` blev løst under integrationen. Den endelige C++-applikation kunne bygges med motor-, netværks-, LiDAR-, encoder-, navigation- og konfigurationsmoduler. Encoderne blev koblet til C++ gennem `libgpiod` på GPIO24 og GPIO25. Før denne rettelse stod positionen stille i hovedprogrammet, selv om Python-testen viste, at hardwaren virkede. Årsagen var, at C++-programmet endnu ikke modtog GPIO edge-events. Efter integrationen opdaterede positionen sig løbende i centimeter.

Navigationen blev testet i et logisk 1 x 1 meter område med et 10 x 10 gitter. Positionen beregnes relativt ud fra encoderafstand og heading. En fejl i grænsekontrollen blev fundet, fordi positionen først blev begrænset til 0-100 cm og derefter kontrolleret for værdier uden for området. Grænsekontrollen blev

ændret, så 0 og 100 cm selv tæller som grænse. Derefter reagerede robotten ved kanten og ændrede retning. Under testen blev der også registreret et checkpoint ved 20 % dækket område.

4. Sikkerhedsanalyse

Der er både elektriske, mekaniske og netværksmæssige risici i prototypen. Raspberry Pi'en må ikke forsynes direkte fra robotbatteriets cirka 6 V. Pi'en har derfor sin egen korrekte strømforsyning, mens motorerne forsynes gennem L9110S-driveren. Fælles GND mellem relevante dele er nødvendig for stabile signaler.

Batterispændingen må heller ikke forbindes direkte til Arduinoens analoge indgang uden hensyn til spændingsniveauet. Derfor bruger vi en 10 kΩ / 10 kΩ spændingsdeler. De midlertidige prototypeforbindelser og taped samlinger er acceptable til test, men ville skulle erstattes af mere robuste stik, isolering og mekanisk fastgørelse i et færdigt produkt.

Motorerne kan bevæge robotten uventet, hvis en GPIO bliver stående HIGH. Derfor bruges en aktiv stopfunktion. LiDAR-logikken registrerer en forhindring inden for cirka 30 cm og udløser stop, bakning og retningsændring. Batterilogikken stopper normal drift ved 20 % eller lavere. Batteriprocenten er dog kun en prototypeindikator, fordi spændingen falder tydeligt under motorbelastning.

MQTT-kommunikation over Wi-Fi giver også sikkerhedsrisici. En prototypebroker uden stærk adgangskontrol bør kun bruges på et kontrolleret netværk. I en rigtig løsning ville vi blandt andet overveje brugergodkendelse, begrænsning af adgang til topics, krypteret kommunikation og validering af kommandoer, så uvedkommende ikke kan sende START eller STOP til robotten.

5. Datastrøm

Vi forsøger ikke kun at gemme rå sensorværdier. Målingerne bliver behandlet til information, som er lettere at forstå og bruge i robotens logik.

Kilde	Rådata	Behandling	Brugbar information
Hjulencoder	Pulser	Pulser + hjuldiameter	Kørt afstand / relativ position
LD06 LiDAR	Afstand i mm + vinkel	Frontsektor og grænse på 300 mm	Forhindring / fri bane
Batteri / Uno R4	Analog A0-værdi	Spænding -> procent	Batteri 0-100 % og low-battery status
MQTT kommando	Tekstpayload	START/STOP fortolkes	Robotstatus / styring
Navigation	Afstand og retning	Position og heading opdateres	Forståelig robotposition/status

Et eksempel er batteriet. A0-værdien er ikke særlig nyttig for en almindelig bruger. Derfor omregnes den først til spænding og derefter til en procent. Procenten sendes som MQTT-data, så systemet kan vise eksempelvis 80 % i stedet for en rå ADC-værdi på cirka 616.

På samme måde bliver LiDAR-data behandlet. Robotten behøver ikke reagere på alle rå punkter på samme måde. Vi fokuserer på området foran robotten og bruger 30 cm som en praktisk grænse for automatisk reaktion.

Den samlede datastrøm er sensor/aktuator -> lokal behandling på robot/Arduino -> MQTT -> server/database/status. Robotten publicerer telemetry og modtager START/STOP via MQTT, mens Arduino Uno R4 publicerer batteriniveau på robot/battery. Encoderpulser omregnes til relativ position og heading, LiDAR-data filtreres til en forhindringsstatus, og navigationen beregner gitterdækning/checkpoints. Server- og databasefunktionerne var en del af projektets kodebase, men vi nåede ikke at gennemføre en

fuld afsluttende database- og offline-test inden afleveringen. Derfor beskriver vi ikke FK6, FK7 og FK10 som dokumenteret bestået.

6. Test og fejlfinding

Vi har forsøgt at teste systemet i små dele, før delene blev samlet. Det gjorde det lettere at finde ud af, om en fejl kom fra hardware, software eller integration. Motorer blev først testet alene, derefter encodere, LiDAR, autonom adfærd, MQTT og batterimåling.

Udvalgte fejl og løsninger

Problem	Fejlfinding/løsning	Resultat
Encoder gav konstant LOW	GPIO og sensor blev testet separat. Sensoren blev udskiftet.	Ny sensor gav korrekte HIGH/LOW-pulser.
PinInvalidState i gpiozero	Floating input manglede active_state. active_state=True blev tilføjet.	Encoderprogrammet kunne køre.
Afstandskørsel overskød på ét hjul	Hver motor blev ændret til at stoppe ved sit eget pulsmål.	Mere ensartet stop omkring det ønskede mål.
Batterimåling ændrede sig næsten ikke	Vi fulgte ledningerne og fandt, at målingen var taget fra motor A/B i stedet for batteriet.	A0 blev stabil omkring 616-618 og gav cirka 6,0 V.
mosquitto_sub: command not found	mosquitto-clients blev installeret på Raspberry Pi.	Pi'en kunne modtage robot/battery over MQTT.
MQTT state=-2 ved første Arduino-opstart	Klienten forsøgte automatisk igen.	Næste forsøg forbandt korrekt.
C++ position stod stille i hovedprogrammet	Python-test viste fungerende encodere. Vi fandt, at C++ endnu ikke modtog GPIO edge-events. libgpiod blev installeret og integreret.	GPIO24/GPIO25 blev aktive i C++, og positionen opdaterede løbende.
Encoder GPIO kunne ikke reserveres	Gamle robot-/encoderprocesser blev stoppet, så GPIO-linjerne blev frigivet.	Encoderne kunne reserveres igen.
Grænsekontrol reagerede ikke korrekt	0 og 100 cm blev gjort til gyldige grænseværdier i stedet for kun at teste værdier udenfor området.	Robotten registrerede kanten og ændrede retning.
LiDAR gav forkerte vinkler	LD06-pakkens byte-layout blev rettet: startvinkel 4-5, målepunkter 6-41 og slutvinkel 42-43.	Vinklerne blev realistiske, og frontområdet kunne kalibreres.
LiDAR gav ekkoer fra robotten selv	Målinger under 120 mm blev filtreret væk, og mindst to hits blev krævet.	Færre falske forhindringer og stabil registrering omkring 17-20 cm.
SSH-forbindelse faldt under en build	Pi blev kontrolleret med temperatur og throttling. Temperaturen var ca. 42,4 C og throttled=0x0. Build blev gentaget med mindre belastning.	Efterfølgende build lykkedes.

Datakvalitet

Encoderdata varierede blandt andet på grund af sensorplacering og mekanisk montering. Vi gentog derfor kalibreringen flere gange og endte med cirka 22 pulser pr. omdrejning på begge hjul. I den afsluttende C++-test blev GPIO24 og GPIO25 læst med libgpiod, og positionen blev opdateret løbende. Resultatet er brugbart til prototypens relative position, men er ikke præcis odometri.

Batteriprocenten svingede kraftigt under motorbelastning, eksempelvis fra over 90 % til omkring 40 % og derefter op igen. Det skyldes spændingsfald under belastning og den simple lineære procentmodel. Vi observerede også, at low-battery logikken kunne udløse sikkerhedsstop ved lave værdier. Batteriprocenten skal derfor betragtes som en prototypeindikator og ikke en præcis SOC-måling.

LiDAR-fejlfinding blev en vigtig del af sluttesten. Den første parser brugte forkerte byte-offsets til vinklerne, hvilket gav forkerte vinkelområder. Efter rettelser blev målingerne realistiske. Vi filtrerede korte ekkoer under 120 mm væk, brugte et kalibreret frontområde på cirka 130-175 grader og krævede mindst to hits. Den afsluttende test registrerede gentagne forhindringer omkring 17-20 cm og udløste autonom reaktion.

Test i forhold til krav

Kravområde	Status ved rapporttidspunktet	Bemærkning
Motorstyring (FK1)	Bestået	C++-test: frem, tilbage, venstre, højre og stop fungerede fysisk.
Autonom kørsel (FK2)	Bestået på prototype	Robotten kørte autonomt i 1 x 1 m testområde og reagerede ved 0/100 cm grænser.
Position/retning (FK3)	Bestået på prototype	C++/libgpiod læser begge encodere; X/Y og heading opdateres løbende.
Obstacle (FK4)	Bestået	LD06 registrerede gentagne forhindringer ca. 17-20 cm foran; robotten stoppede, bakkede og drejede.
Trådløs kommunikation (FK5)	Delvist dokumenteret	MQTT virker til batteridata samt START/STOP. Fuld serverkæde blev ikke sluttet.
Database (FK6)	Ikke sluttet	Kode/serverdel findes i projektet, men fuld database-logning blev ikke dokumenteret i sluttesten.
Serverstatus (FK7)	Ikke sluttet	Afhænger af den server/database-del, som ikke nåede fuld afsluttende test.
Rute/checkpoints (FK8)	Bestået på prototype	10 x 10 gitter og relativ navigation; checkpoint ved 20 % dækning blev registreret.
Batteri (FK9)	Bestået som prototypefunktion	Uno R4 sender 0-100 % via MQTT, og lav værdi kan udløse sikkerhedsstop. Målingen er belastningsfølsom.
Offline efter 10 sek. (FK10)	Ikke sluttet	Kravet blev ikke dokumenteret med en afsluttende 10-sekunders offline-test.
START/STOP (FK11)	Bestået	MQTT START satte robotten i gang, og STOP stoppede motorerne fysisk.

7. Konklusion

Projektet har givet os praktisk erfaring med, hvordan mange små hardware- og softwaredele skal fungere sammen i en embedded/IoT-løsning. Det vigtigste har ikke kun været at få en motor eller sensor til at virke alene, men at integrere motorer, encodere, LiDAR, navigation, MQTT og batteriovervågning i den samme prototype.

Vi har især lært, at systematisk fejlfinding er nødvendig. Flere problemer viste sig at have andre årsager end vi først forventede. Et eksempel var encoderen, hvor problemet viste sig at være en defekt sensor. Et andet var batterimålingen, hvor softwaren så korrekt ud, men måleledningen var taget fra det forkerte sted på motorcontrolleren.

Vi har også fået erfaring med Git/GitHub og med at arbejde flere personer på samme projekt. Marks sygdom i den første uge ændrede arbejdsfordelingen og gjorde tidsplanen strammere. Da han kom tilbage, arbejdede vi mere parallelt. Det gjorde samtidig integration og tydelig opgavefordeling vigtigere.

Prototypen er ikke en færdig robotplæneklipper, men de vigtigste robotfunktioner blev demonstreret: motorstyring, autonom kørsel i det definerede testområde, relativ position/retning, LiDAR-baseret forhindringsreaktion, checkpoints, batteridata og MQTT START/STOP. Server/database og offline-detektion blev ikke fuldt sluttet inden afleveringen og står derfor som uafsluttede dele. Videreudvikling kunne være mere præcis navigation, bedre batterikalibrering, robust mekanisk montering, færdig server/database-test og senere automatisk returnering til ladestation.

Erfaringerne kan bruges senere i uddannelsen og til svendepreven, fordi projektet kombinerer programmering, Linux, hardware, sensorer, netværk, IoT-protokoller, fejlfinding og versionsstyring. Den samme måde at opdele et større system i mindre testbare moduler kan også bruges på en læreplads eller i et fremtidigt job.

Afsluttende status

Den sidste samlede test bekræftede, at robotten kunne bygges og starte med konfiguration, LiDAR, encodere og MQTT aktive. MQTT START/STOP virkede fysisk, position og heading blev opdateret, grænserne blev registreret, og LiDAR registrerede gentagne forhindringer og udløste autonom reaktion. De dele, vi ikke nåede at dokumentere med en afsluttende test, er markeret tydeligt som ikke sluttet i tabellen ovenfor.